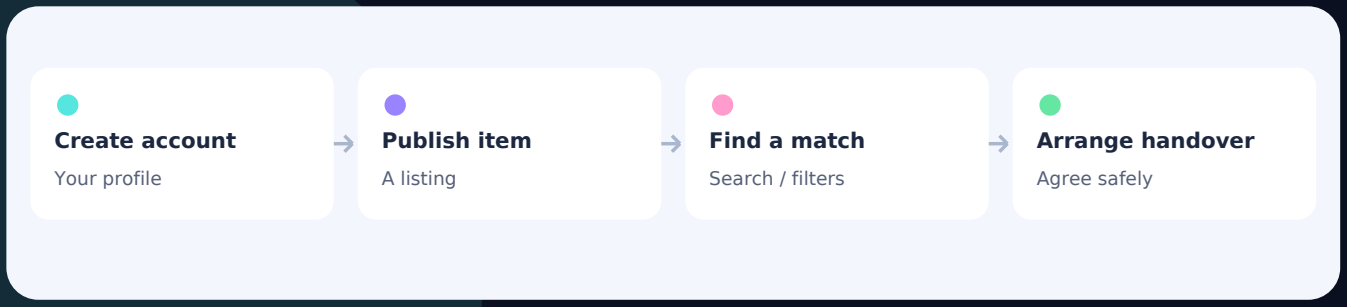
U

UNITRADE

Campus circular marketplace

Trade smarter.
Reuse better.

A beginner-friendly guide to UniTrade, clean data, Google sign-in, Firebase, database security, and public deployment.



Website: https://unitradeneo.vercel.app/

Written in plain English. No previous programming or database experience assumed.

1. What UniTrade does

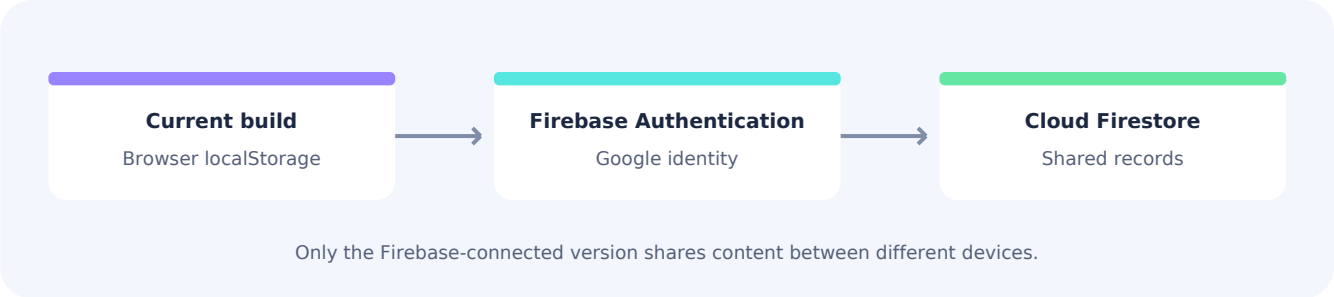
UniTrade is a campus-focused marketplace. It helps students find items nearby and lets them offer things they no longer need. Instead of buying every item new, students can buy, rent, borrow, swap, or collect free items from their campus community.

The problem it solves

A new student may need a calculator, lab coat, drafting board, kettle, cycle, mattress, or books. A senior student may have the same item sitting unused. UniTrade is designed to help those people discover each other and arrange a local handover.

What the current clean release means

The published page is a frontend website. It uses HTML to describe the page, CSS to control appearance, and JavaScript to make controls work. At the moment, the saved records stay in the browser that created them. Firebase has not yet been connected, so another visitor on another device will not automatically see those records.



This distinction matters: a website can be publicly visible on the internet while its data is still local to one browser. Shared data requires the JavaScript to call a cloud database.

Why start with a clean catalog?

The clean release does not create example listings, requests, reports, resource cards, or Karma points. On the first clean load, those areas should be empty until somebody adds real information. It also removes known old seeded records from the browser when the new version starts. It does not erase every real user-created record automatically.

2. Understand each major website area

Home: explains what UniTrade is and gives quick access to the main areas. Marketplace: shows items that people have listed. Looking For: lets a student post what they need. Community: shows personal Karma and useful giving features. Services: groups the specialist tools so the homepage does not become one long page.

Rent & Borrow

A student can offer an item for a short period instead of selling it. A rental can include dates, price, deposit and return expectations. Borrowing can be free. Before a real reservation system is connected, users still need to agree carefully on availability and handover.

Barter & Trade

Students can propose an exchange: for example, a lamp for a bookshelf. A trade offer should identify the two items, explain the offer, and record a status such as pending, accepted, declined or cancelled. A production version must ensure only the sender and intended recipient can see the private trade.

Starter Kits, Fire Sale and Vacation Hold

Starter Kits group a student’s own relevant items into one bundle idea. Fire Sale surfaces a user’s own items for quicker semester-end handover. Vacation Hold is for short-term storage or holding arrangements. The clean release leaves these sections empty until genuine records exist.

Digital Vault

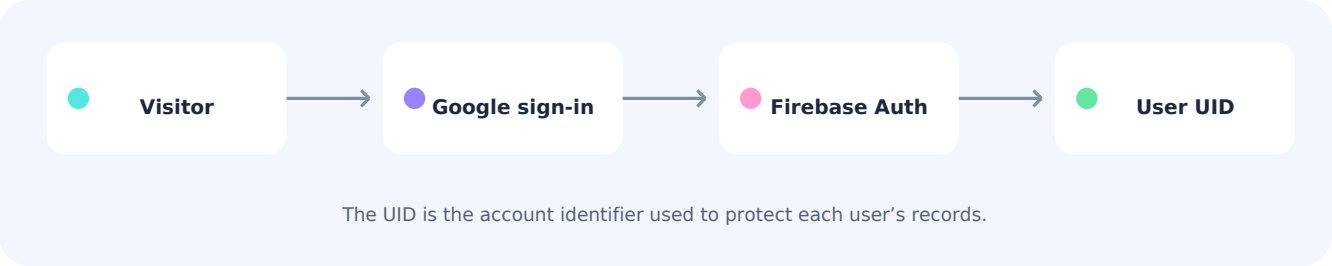
The Vault can describe authorised learning resources. The current data model can store titles and subject information, but real PDF uploads need cloud storage and permissions. Only share materials you own or have permission to distribute; do not upload copyrighted textbooks without authorisation.

Trust & Safety

Use visible public handover areas, inspect an item, keep a record of agreements, avoid sharing OTPs and passwords, and use reporting tools. UniTrade cannot guarantee every transaction is safe.

3. Sign-in, profiles, and account data

Sign-in answers “Who is using the website?” A real product should use a dedicated identity service rather than storing password records in browser storage. Google sign-in lets a person prove control of a Google account; it does not by itself prove that the person is a student at a particular college.



The UID is the account identifier used to protect each user’s records.

A Firebase Authentication user receives a unique user ID (UID). UniTrade can use that UID as the owner of a listing, request or report. That is more reliable than deciding ownership from a display name, because two people can have the same name.

Profile data

A profile may include display name, email supplied by the identity provider, profile photo, college, course, year and a broad hostel/block label. Keep private information private. Do not put phone numbers, student ID numbers, passwords or private messages in a public listing.

What changes when Firebase is added?

The current local account list and password hashes are not automatically converted into Firebase accounts. A person will need to sign in through the new authentication provider. Any real old listing that needs to be kept must be reviewed and deliberately migrated; it is not automatically uploaded.

Important limitation

Do not open public registrations for real transactions while the site still presents localStorage accounts as if they were a real shared account system. Connect Firebase Authentication and move shared listings/requests to Firestore first.

4. Create your Firebase project

Step 1 - Open the Firebase Console

Visit https://console.firebase.google.com/ and sign in with Google. Choose Add project, give the project a clear name such as “UniTrade Production”, and complete the steps. Analytics is optional at the beginning.

Step 2 - Register a website app

On the project overview screen, click the Web icon shown as . Give the app a name such as “UniTrade Web”. Firebase will show a configuration object containing project fields such as apiKey, authDomain, projectId, storageBucket, messagingSenderId and appId. These browser configuration values identify the Firebase project; they are not a service-account private key.

Step 3 - Enable Google sign-in

Open Build > Authentication. Select Get started if displayed, then Sign-in method or Sign-in providers. Choose Google, enable it, select a support email and save.

Step 4 - Authorise your Vercel domain

In Authentication settings, find Authorized domains and add the exact hostname below. Do not include https:// or a slash:

unitradeneo.vercel.app

Also add localhost if you will test using a local web server and it is not already authorised. Do not use Google sign-in from file:// pages; use localhost or the deployed HTTPS website.

Official instructions: https://firebase.google.com/docs/auth/web/google-signin

5. Create Cloud Firestore

Firestore is the shared cloud database. Think of it as a set of online folders called collections. Each collection contains small documents (records). The site JavaScript will create, read, update and delete those documents through the Firebase SDK.



Suggested collections

users/{uid} - a private profile.
listings/{listingId} - publicly safe item information and ownerUid.
requests/{requestId} - an open item request and broad location label.
lostFound/{reportId} - only safe item details; avoid ID numbers or sensitive proof.
trades/{tradeId} - a private exchange between senderUid and recipientUid.
vaultItems/{resourceId} - resource metadata and a cloud-storage path.
users/{uid}/notifications/{id} - private notifications.

Create the database

Open Build > Firestore Database and choose Create database. Start in production mode. Choose a region close to your expected user community and think carefully, because moving a database later is not a simple switch. Paste and adapt the supplied firestore.rules file only after testing the fields and reads/writes that your JavaScript will perform.

Do not use open rules

Never publish rules that say allow read, write: if true. Such rules can let anyone access data. A signed-in user should not automatically be allowed to edit someone else’s listing. Each write should check that ownerUid matches the signed-in user UID.

Official setup: https://firebase.google.com/docs/firestore/quickstart

6. Connect the website code to Firebase

Important: creating a Firebase project does not connect this website automatically. You must replace the current localStorage actions for shared data with real Firebase Auth and Firestore calls.

A. Initialise Firebase

The SDK uses JavaScript modules. The current three-file version can use browser-module imports, though Firebase recommends a module bundler for most production apps. When you choose browser modules, change the script tag to type="module" and add the Firebase imports at the top of script.js.

```
import { initializeApp } from
'https://www.gstatic.com/firebasejs/12.19.0/firebase-app.js';
import { getAuth, GoogleAuthProvider } from
'https://www.gstatic.com/firebasejs/12.19.0/firebase-auth.js';
import { getFirestore, collection, addDoc, getDocs, query, where,
doc, updateDoc, deleteDoc, serverTimestamp } from
'https://www.gstatic.com/firebasejs/12.19.0/firebase-firestore.js';

const app = initializeApp(firebaseConfig);
const auth = getAuth(app);
const db = getFirestore(app);
const provider = new GoogleAuthProvider();
```

This is an example initialisation shape. Replace firebaseConfig with the real values from your Firebase Web app. Do not paste a Firebase Admin private key into client code.

B. Add Google sign-in

Create a Google button that calls signInWithPopup(auth, provider) on desktop; on mobile, Firebase notes a redirect flow may be preferable. Use onAuthStateChanged(auth, user => ...) to restore the signed-in state. Use signOut(auth) to log out. Let Firebase handle credentials; do not keep password hashes in localStorage for the real public account system.

C. Replace the data functions

When publishing a listing, call addDoc(collection(db, "listings"), { ownerUid: user.uid, title, description, price, mode, status: "active", createdAt: serverTimestamp() }). To browse, query the listings collection with where("status", "==", "active"). Then render the returned documents. Edit with updateDoc and remove with deleteDoc. Repeat this migration for requests, Lost & Found, Vault metadata and trade offers.

7. Security rules and personal information

Security rules are the locks on the database. The code in the browser is not a trusted security boundary because visitors can inspect and change it. Firestore rules must decide which user can read or write each record.

What the included rules aim to do

- A user can create and update their own private profile.
- Anyone can read a listing only when its status is active, but only the owner can change or delete it.
- Open Looking For requests are discoverable without exposing a phone number or student ID.
- Lost & Found reports require sign-in and should contain no sensitive identifying details.
- Trades are readable only by the sender and intended recipient.
- Vault file access remains private until an authorised sharing/payment flow is built.
- All collections not explicitly allowed are denied.

Rules and queries must agree

For example, the listing rules in firestore.rules expect a query to include status == active. If the query does not match what the rules allow, Firestore will deny it. Test with two separate Google accounts: the first should be able to edit its own listing, and the second should not be able to edit or delete it.

Never put private data in a public listing

Do not put email addresses, phone numbers, full student IDs, passwords, private messages, identity-document photos, or exact sensitive locations in publicly readable documents. Store private profile details separately. Treat Lost & Found reports carefully: do not disclose the full number or unique details from an ID card in the public description.

The included rules are a secure starting point, not a claim of a completed security audit. Adapt the rules to the final schema and test them in the Firebase Emulator or Rules Simulator before launch.

Official rules: https://firebase.google.com/docs/firestore/security/rules-structure

8. Upload images and study files safely

Firestore stores the information about a listing, but images and PDFs should live in file storage. Firebase Storage can store the actual file, while Firestore stores a storage path and basic metadata such as file type, size, owner UID and date.

The supplied storage.rules file

It provides a starter pattern: listing photos are readable publicly but can only be uploaded or deleted by the signed-in user who owns that folder; images have a small size limit and allowed image types. Vault files are private to their owner. This intentionally does not unlock private or paid study-file downloads.

Before using file uploads

Check the current Firebase pricing and whether your account must enable billing for the storage features you choose. Add a file-size limit, validate the content type, reject unexpected extensions, and provide a way to report copyrighted or harmful material. A file name ending in .pdf does not prove it is a safe or authorised PDF.

What is still not active automatically?

Secure chat, real payment splitting, paid Vault access, institutional student verification, realtime notifications, and moderation tools need their own implementation. Do not show a successful payment status just because a browser button was clicked. Real payments require a supported payment gateway and trusted server-side confirmation.

Privacy basics

Publish a Privacy Policy and Terms of Service before inviting the public. Explain which data is stored, why it is collected, who can see it, how users report content, and how they request account deletion. Store only information required for the product to work.

9. Publish the new version to Vercel

Your URL is already hosted at https://unitradeneo.vercel.app/. To replace the current deployment, update the files in the GitHub repository connected to that Vercel project, then push a commit to the production branch. Vercel will normally create a new deployment automatically.

Step-by-step

1. Download and extract the clean ZIP.
2. Back up your GitHub repository before replacing files.
3. Replace index.html, style.css and script.js in the project folder. Keep the PDF guide alongside them so Product Guide links work.
4. Add firestore.rules and storage.rules to your repository for version control; publishing these files to GitHub does not itself publish them to Firebase.
5. Commit and push the changes.
6. In Vercel Dashboard, open the UniTrade project and wait for the latest deployment status to become Ready.
7. Open the live URL in a private/incognito window and confirm the sample listings, requests and Karma values are gone.

Test before inviting users

- Sign in with Google from the exact Vercel hostname.
- Create a listing using account A.
- Open another browser or device and sign in with account B.
- Confirm the active listing appears there after Firestore is connected.
- Confirm account B cannot edit/delete account A’s listing.
- Confirm a signed-out visitor cannot post a listing.
- Confirm reloads preserve cloud data.
- Confirm that old localStorage accounts are not presented as Firebase accounts.

If the new page still shows old sample activity, hard-refresh and inspect the Vercel deployment commit. The deployed code must be the clean files, and the browser must load the updated script.js.

10. What needs to happen before public launch?

Use this checklist in order. Do not skip database security to make the page “work”; open rules can expose or destroy your data.

Before inviting the public

- ☐ Firebase Authentication is connected to the sign-in UI.
- ☐ Google provider is enabled and unitradeneo.vercel.app is authorised.
- ☐ All shared listing/request/report operations use Firestore, not only localStorage.
- ☐ Security rules are tested with two different accounts.
- ☐ Storage rules are published and tested if users can upload files.
- ☐ Privacy Policy and Terms are linked in the footer.
- ☐ Users can report listings and there is a real moderation response process.
- ☐ Account deletion and personal-data removal are planned.
- ☐ Firebase usage/budget alerts and backup/export plan are configured.
- ☐ No unsupported claim says “100% safe” or “guaranteed scam-free”.
- ☐ Payments and secure chat are not advertised until connected and tested.

A note about existing data

localStorage is tied to a specific browser profile. It does not move to Firebase by itself. This clean release removes known seeded records but keeps non-seed records that are stored locally. If you have genuine personal records you want to preserve, inspect them before using Clear Local Data. A deliberate migration should map records to real Firebase UIDs and follow the published rules.

A note about this package

The website files in this release still need the Firebase configuration and Firestore integration steps above. This handbook gives the implementation path and starter rules; it does not claim that Google Authentication or cloud syncing has already been enabled.

11. Glossary for beginners

Frontend: the parts people see and click, such as buttons, cards, forms and navigation.
HTML: the structure and text of the web page.
CSS: the colours, spacing, layout and motion of the page.
JavaScript: the logic that makes controls react and saves or loads information.
localStorage: a small storage area inside one browser. It is not shared with other devices.
Firebase Authentication: a service that signs users in and provides a verified account identity for the app.
Google sign-in: a way for a person to sign in with a Google account. It does not automatically prove college enrollment.
Cloud Firestore: the database used to share records between devices and users.
Collection: a group of related records, similar to a folder.
Document: one record inside a collection, such as one listing.
UID: a unique user ID provided by Firebase Authentication.
Security rules: database permissions that decide who can read, create, edit or delete records.
Vercel: the hosting service serving the public website URL.
Cloud Storage: a place to store actual images and files rather than only their text descriptions.

Official references

Firebase web setup: https://firebase.google.com/docs/web/setup
Google sign-in: https://firebase.google.com/docs/auth/web/google-signin
Firestore quickstart: https://firebase.google.com/docs/firestore/quickstart
Firestore rules: https://firebase.google.com/docs/firestore/security/rules-structure
Vercel: https://vercel.com/docs

UniTrade
Trade smarter.
Reuse better.

First clean the data. Then connect identity. Then secure the database. Finally test it with two accounts before inviting the campus community.